

APEX: Five Open Technical Risks — Decision Brief for the 12-Day Build

TL;DR

- **Draw two hard architectural seams:** gradient flow stops at the cvxpylayers SCP projector; everything downstream (Mellea Instruct-Validate-Repair, Granite Guardian, Chronos-2 API, agent prompts) is optimized **outside** the gradient with DSPy/GEPA reflective prompt evolution — do **not** try to estimate gradients through LLM text.
 - **The offline browser path is feasible but must be radically scoped:** ship Granite 4.0 Nano via Transformers.js + ONNX Runtime Web/WebGPU, do the friction-ellipse projection as a 30-line Newton root-find in JS/WGSL (sub-microsecond), and treat the offline summary as a **server-authoritative draft** with a CRDT-style reconciliation contract so it can never contradict the post-sync coaching report.
 - **For safety-critical realism:** encode FIA Certificate-of-Adaptations flags as a **strict lexicographic constraint hierarchy** with elastic slacks at each level; ship APEX-Bench with a Docker-pinned, FP32-deterministic harness and **tolerance bands plus 95% bootstrap CIs** (not bit-exact scoring); and add an explicit "**physics-model-may-be-wrong**" detector built from Granite TSPulse anomaly score + conformal residual interval over the projection layer's predicted vs. actual telemetry.
-

Key Findings

- The graph cannot, and should not, be made fully end-to-end differentiable. The right architectural move is a two-regime decomposition with the seam at the SCP projector output.
- A 2-D friction-ellipse projection is a closed-form scalar Newton root-find, ~50-200 ns per call in JavaScript — there is no need to ship a general QP solver to the browser for the offline path, though several named WASM-portable solvers exist (`scs-solver` npm, Clarabel.rs `wasm` feature, DAQP, CVXPYgen+Emscripten) as fallbacks.
- Lexicographic / preemptive goal programming with separated-order-of-magnitude slack penalties (as in Quan, Jeddi, Prignoli, Falcone 2025, "Priority-driven Constraints Softening in Safe MPC", arXiv:2503.15373) is the standard tool for FIA-COA-style hierarchical constraints in MPC and matches APEX exactly.
- Bit-exact reproducibility across GPU types is impossible (PyTorch's own docs disclaim it); the credible benchmark protocol is FP32 + pinned Docker + tolerance bands + bootstrap CIs, modeled on MLPerf Tiny.

- A "physics-confidence" upstream signal — combining IBM Granite TSPulse anomaly score, conformal residual intervals, and a Mahalanobis distance on the TTM embedding — can be passed as a BYOC field into Granite Guardian 4.1-8B, which is explicitly trained (April 2026 release) to faithfully judge complex multi-part user-defined criteria.
-

Details

Question 1 — End-to-End Gradient Flow Across the Continuous-Discrete Divide

Recommended position: do NOT try to make the graph end-to-end differentiable. Draw a clean two-regime boundary at the output of the SCP projector.

Regime A — Differentiable inner loop (joint gradient descent):

- TTM r2.1 channel-mix fine-tuning head (the only learnable weights inside the gradient zone)
- 3-iteration unrolled SCP physics-projection layer, implemented with `cvxpylayers` (the canonical NeurIPS 2019 library by Agrawal, Amos, Barratt, Boyd, Diamond, Kolter; install: `pip install cvxpylayers`; CuClairabel backend for GPU). `cvxpylayers` solves a DPP-form parametrized convex program in the forward pass and computes derivatives in the backward pass via implicit differentiation of the KKT conditions — exactly what the unrolled SCP needs.
- Loss: physics-violation rate + forecast MSE vs. ground truth telemetry on held-out laps.
- Trained on the RTX 4060 with small batch sizes; the M2 is for inference/demos.

Regime B — Non-gradient outer loop (reflective prompt optimization):

- Mellea Instruct-Validate-Repair text generation (the report writer)
- The tri-agent Agent-as-Judge critic (Physics-Critic / Pedagogy-Critic / Granite Guardian)
- All LangGraph node prompts and the MCP tool-use descriptions
- Optimized via **DSPy** + **GEPA** (Agrawal et al., "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning", arXiv:2507.19457, accepted as an ICLR 2026 Oral, last revised 14 Feb 2026; 17-author team across UC Berkeley, Stanford, MIT, and Databricks, led by Lakshya A Agrawal and Omar Khattab; install: `pip install dspy gepa`).

GEPA is the correct tool here because:

1. It treats the entire critic loop as a textual program and evolves the prompts using LLM reflection on full execution traces (error messages, critic verdicts), not just scalar rewards.
2. It maintains a Pareto frontier across the three critic dimensions (physics, pedagogy, safety), avoiding mode collapse to one critic.

3. The ICLR 2026 paper reports: "Across four tasks, GEPA outperforms GRPO by 10% on average and by up to 20%, while using up to 35x fewer rollouts." That is the rollout efficiency that makes it tractable in a 12-day build budget.

Why NOT to use gradient estimators across the seam. Score-function / REINFORCE estimators (Williams 1992) and Gumbel-Softmax / Straight-Through (Jang/Maddison 2017) exist for backpropagating through discrete sampling. STGS dominates discrete-latent VAEs and NAS. For APEX they are the wrong tool:

- The discrete output is a multi-sentence natural-language report, not a categorical latent of size K. Vocabulary-sized Gumbel-Softmax is high-variance and empirically unstable for long sequences.
- REINFORCE on a multi-token sequence with a sparse Guardian "good/bad" reward has variance that explodes with sequence length; control variates (RELAX, REBAR) need a learned baseline that is itself non-trivial.
- Chronos-2 is a third-party API, so there is no parameter to backprop into; differentiable surrogate-model approaches (e.g., Shirobokov et al. 2020, "Black-Box Optimization with Local Generative Surrogates", arXiv:2002.04632) would require training a surrogate of Chronos-2, which is out of scope.

Concrete architectural seam:

```
[Telemetry input]
  → TTM (frozen body, fine-tuned head)
  → FlowState (frozen)
  → Chronos-2 API (black-box)
  → cvxpylayers SCP projection (3 unrolled iterations)
  ─────────── ▽ STOPS HERE ───────────
  → Textual violation log (string)
  → LangGraph orchestration (DSPy/GEPA-optimized prompts)
  → Mellea Instruct-Validate-Repair (RejectionSamplingStrategy, loop_budget=5)
  → Granite Guardian 4.1-8B BYOC verdict
  → Final coaching report
```

Ensemble averaging

Training schedule:

- **Days 1-6:** Train Regime A on RTX 4060. Loss = $\alpha \cdot \text{MSE}(\text{forecast}, \text{ground-truth}) + \beta \cdot (\text{physics-violation-rate from projector}) + \gamma \cdot (\text{distance moved by projector})$. Only the TTM head weights change. Chronos-2 and FlowState contribute frozen predictions to the ensemble; the channel-mix head learns to weight them.
- **Days 7-10:** With Regime A frozen, run DSPy/GEPA on a held-out lap set to optimize the prompts of the three critics and the Mellea repair templates. Metric: composite score =

pedagogical clarity (judge model) + Guardian pass-rate + ground-truth driver-feedback agreement.

- **Days 11-12:** End-to-end evaluation, no further training.

Implementation pitfalls to pre-empt:

- `cvxpylayers` requires DPP form; the eight-tier physics model linearization must be re-expressed as DPP at each SCP iteration (Pacejka linearized about the previous SCP solution, etc.). Budget time for this; it is the single hardest part of Regime A.
- `CvxpyLayer` cannot be `torch.jit.trace`-d. Plan for eager-mode inference.
- Unrolling 3 SCP iterations means the backward pass calls the implicit-diff backward 3 times. On the RTX 4060 with $\sim 14\text{-channel} \times 30\text{-horizon}$ problems, budget $\sim 150\text{-}400\text{ ms}$ forward + $\sim 400\text{-}800\text{ ms}$ backward per sample; use batch size 1-4.

Question 2 — Offline Edge Consistency for the In-Browser WebGPU Path

Recommended position: feasible. Ship a *scoped* offline summary that asserts only what the edge model can actually verify (a 2-D friction-ellipse check), and treat the offline output as a draft that the server overwrites on reconnect.

Why NOT to port the full SCP. Porting the eight-tier Pacejka SCP solver to WebAssembly is theoretically possible (every named solver below has a WASM path) but the wrong engineering call for a 12-day build. The right move is to ship a radically simplified projection that the laptop browser can run instantly.

The named browser-side stack — LLM inference:

- Granite 4.0 Nano (350M parameters, hybrid SSM-Transformer) is the right size. IBM ships an official `onnx-community/granite-4.0-micro-ONNX-web` model and a working WebGPU demo space (`huggingface.co/spaces/ibm-granite/Granite-4.0-Nano-WebGPU`). Use Transformers.js v4 with `device: 'webgpu', dtype: 'q4'`. On an M2 MacBook expect 30-50 tok/s; on a Windows laptop with integrated graphics, fall back to `dtype: 'q4'` on WASM.
- ONNX Runtime Web 1.17+ exposes the WebGPU execution provider; switch with `executionProviders: ['webgpu']`. Falls back to WASM SIMD automatically. Capability detection should be done at session create.

Convex projection in the browser: the friction-ellipse / friction-circle constraint, $\sqrt{a \cdot x^2 + b \cdot y^2} \leq \mu$, has a closed-form scalar Newton solution (Eberly's algorithm; "Distance from a point to an ellipse" by David Eberly, geometrictools.com). Implementing it directly in JavaScript:

javascript

```
function projectFrictionEllipse(ax, ay, a, b) {
  if (a*ax*ax + b*ay*ay <= 1.0) return [ax, ay]; // already feasible
  let lambda = 0.0;
  for (let i = 0; i < 8; i++) { // Newton, ~1e-12 in 4-8 iters
    const dx = 1.0 + 2*lambda*a, dy = 1.0 + 2*lambda*b;
    const x = ax/dx, y = ay/dy;
    const F = a*x*x + b*y*y - 1.0;
    const Fp = -4*a*a*x*x/dx - 4*b*b*y*y/dy;
    lambda -= F/Fp;
  }
  return [ax/(1+2*lambda*a), ay/(1+2*lambda*b)];
}
```

Per-call cost is ~50-200 nanoseconds in V8. For a 60-second lap of 30-Hz telemetry (~1800 samples) this is single-digit milliseconds — orders of magnitude faster than any general QP solver.

General small QPs in the browser (fallback options, sub-100 variables):

- **scs-solver** (npm) — DominikPeters/scs.wasm, well-maintained, supports LP/QP/SOCP/exponential cones, warm-starting. ~1-5 ms per solve. [GitHub](#)
- **OSQP** — npm package [osqp](#) exists but stale (2021, OSQP 0.6-era); for production rebuild OSQP 1.x library-free C with current Emscripten. Sub-millisecond to ~3 ms for sub-100-var QPs.
- **Clarabel.rs** — has an explicit [wasm](#) Cargo feature; `cargo build --target wasm32-unknown-unknown` + `wasm-bindgen`.
- **DAQP** (Arnström) — library-free C, dual active-set, sweet spot is exactly sub-100-var dense QPs; sub-millisecond in WASM. [GitHub](#)
- **CVXPYgen** (`pip install cvxpygen`), paper arXiv:2203.11419 by Schaller, Banjac, Diamond, Agrawal, Stellato, Boyd) emits library-free C from a DPP CVXPY problem; compile that C with [emcc](#)/[emcmake](#). The Light-weight-MPC/MPC-simulator GitHub repo is a working OSQP+Emscripten+JS frontend template.
- A precedent demo for in-browser MPC exists: Gareth Cross, "Model Predictive Control in the browser with WebAssembly" (Oct 2024, cart-pole swing-up, runs real-time on mobile).

Offline-online consistency contract. The browser cannot reach the LangGraph state, the heavy SCP, or the RAG store. Design rule: **the offline summary is allowed to assert ONLY what the edge model can independently verify.** Concretely the offline summary is restricted to:

1. Single-sample friction-ellipse violation count (verified by the in-browser Eberly Newton).
2. Lap-time delta vs. driver's recent personal best (verified by local timestamps).

3. A short, fixed-template Granite Nano paddock note ("You exceeded the friction envelope in 3 corners; the most severe was T7 entry. Server analysis pending.")
4. **No mechanical or strategic recommendations** — those require the full physics layer and RAG context.

Reconciliation pattern (offline → server). Use the standard server-authoritative + CRDT-friendly metadata pattern (per Ditto/Figma/p2panda offline-first guidance: "Hard constraints (payments, inventory, permissions) remain server-authoritative"):

- Each offline summary is stored as an `OfflineDraft` record with `lap_id`, `device_id`, `client_timestamp`, `granite_nano_version`, `physics_simple_version`, a JSON list of `{tier, claim, evidence}` items.
- On reconnect, the server runs the full pipeline. The server then publishes an authoritative `CoachingReport` with provenance pointing back to the `OfflineDraft.id`.
- The frontend **MUST** replace any field of the offline draft with the server version if it exists (last-writer-wins on the server side only; the offline draft never overrides the server). This is precisely the pattern recommended by Calibraint/Ditto for offline-first apps where "two devices reserving the last seat" needs server arbitration.
- For strictly local UI state (notes the driver typed, view preferences), use a Yjs/Automerger CRDT (well-supported in browser TypeScript) so multi-device editing converges without conflict.

Memory and latency budget:

- Granite 4.0 Nano 350M q4 quantized: ~200-300 MB download (cached in IndexedDB), ~400 MB VRAM resident, ~30 tok/s on M2.
- ONNX Runtime Web session init: 1-3 seconds on first load (cached after).
- Friction-ellipse projection over a 60-second lap: <10 ms total.
- Paddock summary generation (200 tokens): ~6-8 seconds end-to-end.

Graceful fallback. If on-device VRAM is insufficient (some older laptops), the system should detect via `navigator.gpu` availability + `navigator.deviceMemory` and fall back to a non-LLM template-filled paddock summary using only the friction-ellipse counts — guaranteed to be physically coherent because it makes no inference.

Question 3 — Resolution of Conflicting FIA Certificate-of-Adaptations Constraints

Recommended position: strict lexicographic constraint hierarchy with elastic slacks at each level, IIS-based diagnostics on hard-infeasibility, and an explicit "safety-fallback report" mode that the driver sees if relaxation is required.

The constraint priority hierarchy (locked, top to bottom):

Tier	Constraint class	Treatment	Rationale
0 (inviolable)	Vehicle kinematic identity (mass, wheelbase, no negative speed)	Hard equality/inequality, NO slack	Violating these means the model has lost physical meaning.
1 (inviolable)	FIA COA hard caps (max steering lock, brake-bias range, throttle/brake complementarity if not MME- disabled)	Hard inequality, NO slack	Legal/safety: the adaptation is the literal certified envelope.
2 (high)	Pacejka friction-ellipse	Elastic slack $\boxed{s_2 \geq 0}$, large penalty $\boxed{\lambda_2 = 10^6}$	Tire model is empirical; a small violation may reflect mis-parameterization, not infeasibility.
3 (medium)	Aerodynamic balance, load- transfer sign	Elastic slack, $\boxed{\lambda_3 = 10^4}$	Modeling assumptions, not certification facts.
4 (low)	Smoothness / regularization	Elastic slack, $\boxed{\lambda_4 = 10^1}$	Soft preference.

This is lexicographic / preemptive goal programming (Tamiz, Mardle, Jones 1995). In cvxpylayers it is implemented by penalizing slack variables with separated orders of magnitude so that level-k slack is minimized completely before level-(k+1) is reduced — a standard elastic-programming pattern. See Quan, Jeddi, Prignoli, Falcone 2025, "Priority-driven Constraints Softening in Safe MPC for Perturbed Systems", arXiv:2503.15373 — a near-perfect match for the APEX use case, including the observation that with appropriately weighted slacks, "whenever a feasible solution to the original hard-constrained problem exists, the optimal solution of the slacked problem coincides with that of the original problem."

Concrete CVXPY pattern:

```
python
```

```

# Tier 0,1: hard
constraints = [kinematic_identities, *coa_hard]    # NO slack added
# Tier 2-4: elastic
s2 = cp.Variable(N, nonneg=True)    # friction
s3 = cp.Variable(N, nonneg=True)    # aero/load
s4 = cp.Variable(N, nonneg=True)    # smoothness
constraints += [
    cp.norm(stack(ax, ay), 2) <= mu*Fz + s2,
    aero_balance_residual <= s3,
    smooth_residual <= s4,
]
penalty = 1e6 * cp.sum(s2) + 1e4 * cp.sum(s3) + 1e1 * cp.sum(s4)
problem = cp.Problem(cp.Minimize(objective + penalty), constraints)

```

Infeasibility-recovery loop (when Tier 0/1 are infeasible):

1. **Detect:** catch `cp.error.SolverError` and check `problem.status in {"infeasible", "infeasible_inaccurate"}`.
2. **Diagnose** with an **Irreducible Infeasible Subset (IIS)** finder. CVXPY does not ship IIS natively, but you can route the problem to Gurobi (free academic license) or implement Chinneck's deletion-filter algorithm (Chinneck & Dravnieks 1991, "Locating minimal infeasible constraint sets in linear programs," ORSA Journal on Computing). The 2023 OptiChat paper (arXiv:2308.12923) demonstrates LLM-driven IIS-based diagnosis; for APEX a non-LLM deletion filter is sufficient. Boyd et al., "Auto-Repair for CVX" (Optimization and Engineering 22:247-259, 2021) is the canonical software pattern.
3. **Recover** in priority order:
 - First, attempt **forecast-side relaxation**: ask the ensemble for a slightly slower, lower-acceleration target (sample a lower-confidence quantile from the TTM forecast). Often the corner is feasible at 95% of forecasted speed.
 - If still infeasible, **never relax Tier 1**. Instead, the projector returns its closest-feasible trajectory (which respects all COA caps) and emits a textual violation-log entry: "Forecasted trajectory infeasible under COA. Returning closest-feasible alternative with energy gap X." Guardian sees this and the report tells the driver: "The car as configured cannot achieve this corner profile at the speed forecast — the analysis is conservative."
4. **Differentiability through infeasibility:** because we never relax Tier 0/1, the path that hits infeasibility falls back to a deterministic projection onto the COA polytope (Dykstra's alternating projections, or a single inner QP that minimizes $\|x - x_{\text{forecast}}\|^2$ subject to COA hard caps only). That fallback IS differentiable in cvxpylayers, so the outer training loop never sees a NaN gradient.

Conflicting COA flags. If a parsed COA file produces two flags that are themselves contradictory (e.g., "throttle-brake complementarity disabled" AND "throttle-brake complementarity enforced"), that is a parse error, not an optimization problem. Surface to the user with the offending COA line numbers; refuse to run until resolved. This is the "two-tier validation" approach (Mellea-style) for the COA parser itself: schema check + semantic check; if the semantic check fails, halt.

Question 4 — Reproducible Cross-Hardware Evaluation for APEX-Bench

Recommended position: tolerance-based scoring with confidence intervals, FP32 reference runs in a pinned Docker container, and explicit "reference" vs "open" submission divisions modeled on MLPerf.

Why bit-exact is impossible. PyTorch's own documentation states bluntly: "Completely reproducible results are not guaranteed across PyTorch releases, individual commits, or different platforms. Furthermore, results may not be reproducible between CPU and GPU executions, even when using identical seeds." Shanmugavelu et al. (arXiv:2408.05148, "Impacts of floating-point non-associativity on reproducibility for HPC and deep learning") quantify the magnitude on a single platform: training GraphSAGE 1,000 times "will produce a unique and non-reproducible set of model weights with each run using identical inputs," with weight variability growing from 1.414 ± 0.05 at epoch 1 to 1.418 ± 0.23 at epoch 10. For LLM inference specifically, Horace He and Thinking Machines Lab ("Defeating Nondeterminism in LLM Inference," Connectionism blog, Sep 2025) showed that sampling 1,000 completions of Qwen3-235B-A22B-Instruct-2507 at temperature 0 produced 80 distinct outputs, all identical for the first 102 tokens before diverging due to non-batch-invariant kernels. With vLLM's PagedAttention plus speculative decoding plus FP8 quantization the situation is worse. Conclusion for APEX-Bench: **stop trying to reproduce bits; reproduce distributions.**

The APEX-Bench reproducibility protocol (recommended, concrete):

1. Reference division — bit-stable within a pinned container.

- Publish a Docker image (e.g., `ghcr.io/apex-bench/reference:v1.0`) pinning CUDA version, PyTorch version (≥ 2.8), vLLM version, cvxpy, cvxpylayers, OSQP/Clarabel, transformers, and the exact OS.
- Reference runs use **FP32 everywhere** (no FP8, no speculative decoding),
`torch.use_deterministic_algorithms(True)`, `CUBLAS_WORKSPACE_CONFIG=:4096:8`,
`torch.backends.cudnn.deterministic=True`, fixed seeds, single-GPU, batch size 1, vLLM
`--enforce-eager` (disables CUDA graphs that batch-vary).
- A submission "passes reference reproduction" if all reported scores match within a tight per-metric tolerance (e.g., MSE within $1e-4$, friction-circle violation rate within 0.5 percentage points absolute).

2. Open division — fast and realistic.

- Allow FP8, speculative decoding, larger batches, sharding.
- Score must be reported as median + bootstrap 95% CI over ≥ 5 independent runs with different seeds. This is the MLPerf Tiny pattern (Banbury et al.): "Report median latency and energy per inference over at least five independent trials."
- A claim is valid only if the 95% CI does not cross the established reference score by more than a published tolerance.

3. The four evaluation axes and their tolerances:

Axis	Metric	Reproduction rule
Accuracy	MSE per channel	Reference: exact. Open: median $\pm$ 95% CI must overlap reference.
Physical compliance	Friction-circle violation rate, complementarity violation rate	Reference: exact (deterministic post-projection counts). Open: tolerance 0.5 pp absolute.
Industrial readiness	p50/p95 latency, peak VRAM, tok/s	Always reported as median + CI.
OOD generalization	Held-out lap performance by track/car/condition	Same pattern as accuracy.

4. Optimization-solver outputs: because SCP is non-convex and the inner QP is convex but solver-dependent, the trajectory can vary slightly across solvers. APEX-Bench should score the projector by:

- **Constraint-satisfaction rate** (exactly reproducible — it's a post-hoc count of how many of the projected samples satisfy each tier).
- **Closeness to reference** measured as RMS distance to the reference trajectory, with a published tolerance band.
- **NOT** the raw trajectory coordinates, which are solver-dependent.

This is analogous to how MLPerf scores the cosine-similarity between embedding outputs rather than demanding bit-exact reconstructions.

5. Submission requirements (modeled on MLPerf rules and MLCommons working-group practice):

- Full Dockerfile + source.

- A `mlcr`-style automation script (the MLCFlow framework from MLCommons is a reasonable model) that one-command runs `mlcr apex-bench --reference` and outputs a JSON results file.
- A compliance checklist: hardware spec, driver version, RNG seeds, exact commit hash, all environment variables.
- Independent partner reproductions are encouraged (per AMD's MLPerf Inference 6.0 pattern: "partner results landed within 4% of submission... and some landed within 1%").

6. Versioning: APEX-Bench v1.0 freezes Granite TTM r2.1, Granite Guardian 4.1-8B, Chronos-2 frozen weights, and a specific seed file. Re-running on a different TTM revision is an "open division" entry, not a "reference reproduction."

What the team must do in 12 days:

- Ship the Docker reference image on day 6 (not day 12) so the team itself can use it for internal regression-checking.
- Wire a `pytest`-based reproducibility CI that runs the reference suite on every commit and asserts the score moves only within the published tolerance.
- Publish a leaderboard scaffold (a simple JSON file in a GitHub Pages repo); submissions are PRs that add a results entry plus a Docker tag.

Question 5 — Guardian Auditing a Physics Model That Could Itself Be Wrong

Recommended position: add an explicit "physics-confidence" upstream signal — a Granite TSPulse anomaly score + a conformal residual check + a Mahalanobis distance on the projection layer's input — and feed all three as numerical fields into Guardian's BYOC criterion, so the safety verdict explicitly conditions on whether the physics is trusted.

The problem stated precisely. Guardian 4.1 BYOC (April 2026, `ibm-granite/granite-guardian-4.1-8b`) trusts the textual violation log. But the violation log is generated by `if friction_circle_violation_rate > threshold: log("over the limit at T7")` where `friction_circle_violation_rate` depends on `mu` (assumed tire-road friction), Pacejka coefficients from a tire property file (TIR), thermal state estimates, etc. If `mu` is wrong by 15% because the track is damp, every claim downstream is wrong.

The named techniques (each appears in the architecture):

A. Residual-based model validation (offline parameter estimation). After each session, compute the residual $r(t) = \text{observed_lateral_accel}(t) - \text{physics_model_predicted_lateral_accel}(t)$ channel by channel. The literature on Pacejka identification (TRIP-ID, Garatti & Bittanti 2009; the homotopy-optimization MF identification work surveyed in IEEE Access; and the 2026 paper "Vision-Augmented On-Track System Identification for Autonomous Racing", arXiv:2603.09399, which uses

MobileNetV3→S4→Nelder-Mead to recover Pacejka parameters from on-track data and "reduces friction estimation error by 76.1% using 85% fewer FLOPs") shows that the cost function $\sum \|r(t)\|^2$ over a clean lap recovers Pacejka parameters within a few percent. APEX runs this post-session to update the driver's tire-property file.

B. Distribution-shift detection (real-time, during the projection).

- **Mahalanobis distance** of the current telemetry window's penultimate-layer embedding (from the TTM body) against the class-conditional Gaussians fitted on the training set. Lee et al. 2018, refined by Ren et al. 2021 (Relative Mahalanobis) and Mueller & Hein 2025 (Mahalanobis++, with L2-normalized features). Threshold by validation-set 99th percentile.
- **Conformal prediction** over the residual $r(t)$: compute non-conformity scores $s(t) = \|r(t)\|_2$ on a calibration set; at session time, an observed residual above the $(1-\alpha)$ -quantile is flagged. The 2024 paper "Out-of-Distribution Detection Should Use Conformal Prediction (and Vice-versa?)" (arXiv:2403.11532) is the right framework: it gives a `conformal AUROC` and `conformal FPR@TPR95` with probabilistic guarantees.

C. Time-series anomaly score on raw telemetry (model-free).

- **IBM Granite TSPulse r1** (ibm-granite/granite-timeseries-tspulse-r1; ~1M parameters, runs on a laptop CPU). Per Ekambaram et al. (arXiv:2505.13033, the TSPulse paper): "TSPulse achieves strong and consistent gains across four TS diagnostic tasks: +20% on the TSB-AD anomaly detection leaderboard, +25% on similarity search, +50% on imputation, and +5-16% on multivariate classification, outperforming models that are 10-100X larger on over 75 datasets." It needs ~1536-2048 samples of context (3-4× its 512 base window). At 30 Hz that's a ~50-70 second telemetry window — perfect for a lap-level audit.
- TSPulse flags pure data anomalies (sensor dropouts, tire-pressure step-changes, surface transitions) that the physics model has no way to know about.

D. Online parameter identifiability. For μ specifically, a recursive least-squares estimator with a forgetting factor 0.98 fed by the lateral-force balance equation produces an online $\mu_{\text{hat}}(t)$ with 95% CI. If $\mu_{\text{hat}}(t)$ deviates from the COA-default by more than 2σ for $>5s$, raise the physics-uncertainty flag.

The integrated detector ("physics-may-be-wrong" signal). Combine into a single scalar `physics_confidence` $\in [0,1]$:

Calibrate the weights so `(physics_confidence < 0.5)` on the held-out "wrong-tire" or "wet-track" laps in your test set.

Feeding the signal into Guardian (BYOC). Guardian 4.1 BYOC criterion is the right plumbing. Per the official IBM Granite Guardian 4.1 documentation: "Granite Guardian 4.1 8B introduces improved Bring Your Own Criteria (BYOC) support, enabling users to define arbitrary judging criteria... The model can now faithfully evaluate complex, multi-part requirements such as formatting rules, length constraints, and domain-specific instructions." The BYOC block is updated to:

```
### Criteria:
```

```
A coaching claim is considered SAFE if and only if:
```

```
(1) it is consistent with the violation log AND
```

```
(2) the physics_confidence score for the session is ≥ 0.5 OR the claim is  
explicitly hedged with the phrase "based on the model assumptions, which may not  
match this session."
```

The session-level `(physics_confidence)` is passed as a numeric field in the Guardian input document.

The Mellea repair loop closes the loop. If Guardian returns "unsafe" because `(physics_confidence < 0.5)` and the report makes an unhedged claim, the Mellea Instruct-Validate-Repair (with `(RejectionSamplingStrategy(loop_budget=5))`) regenerates the report with the hedge automatically — the same pattern as the Mellea email-formatting example, just with the "physics-may-be-wrong" requirement injected.

What this does NOT solve. It does not solve the case where the physics model is systematically biased in a way that is consistent with the training data — that is, where the wrong physics still produces low residuals on the lap because the wrong parameters happened to fit. The mitigation is the standard one: report ground-truth tire-temperature data from the car (Team BRIT and most grassroots cars now have IR tire temperature sensors) as a side-channel, and validate the thermal sub-model offline against actual measured temperatures. If the offline validation fails, flag the affected tire-property file as deprecated. This is an explicit prototype experiment that should be scoped post-12-day.

Recommendations (Staged)

Day 1-2 — Lock the architecture per this brief.

- Adopt the two-regime gradient/non-gradient seam (Q1 recommendation).
- Adopt the lexicographic priority hierarchy with elastic slacks (Q3 recommendation).
- Choose `scs-solver` (npm) as the fallback general QP solver and JS-Newton friction-ellipse as the primary offline projection (Q2 recommendation).
- Choose Granite TSPulse + conformal residuals + Mahalanobis for the physics-confidence detector (Q5 recommendation).

Day 3-6 — Build Regime A.

- Implement the 3-iteration cvxpylayers SCP projector in DPP form.
- Implement the elastic-slack constraint hierarchy.
- Train the TTM channel-mix head on the RTX 4060.
- Stand up the Docker reference image (Q4) — by day 6, not day 12.

Day 7-10 — Build Regime B and the browser path.

- Wire DSPy/GEPA over the three critics + Mellea repair templates.
- Ship the Transformers.js + ONNX Runtime Web + WebGPU offline path.
- Wire the offline-online reconciliation contract.

Day 11-12 — Evaluate and freeze APEX-Bench v1.0.

- Run the bench harness in reference mode.
- Publish results JSON, leaderboard scaffold, Dockerfile, MLCFlow-style automation.

Benchmarks that should change the plan:

- If cvxpylayers backward exceeds 1 s on the RTX 4060 for a 14-channel, 30-step problem with 3 SCP iterations, drop one SCP iteration (go to 2) or use Clarabel's CuClarabel GPU backend.
- If DSPy/GEPA does not improve composite score after 30 rollouts, reduce the prompt search space (lock the Guardian and Physics-Critic prompts, optimize only Pedagogy-Critic and Mellea).
- If Granite Nano 350M on WebGPU is slower than 15 tok/s on the M2, fall back to the template-filled paddock summary; do not block the build.
- If the FIA COA parser cannot reliably extract complementarity flags, fall back to a configurable JSON COA file the user uploads; do not block.

Caveats

- **DSPy + GEPA on a 12-day budget** is non-trivial; the team should validate the wiring on a toy task (e.g., math reasoning prompt optimization) by day 8 to fail fast.
- **No vendor ships an official OSQP WASM npm package**; the team should plan to rebuild OSQP 1.x with current Emscripten if they need OSQP semantics in the browser — `scs-solver` is the safer default.
- **Granite Guardian 4.1 BYOC** ships April 2026 per the model card; verify it is actually available before locking on the 8B variant. Granite Guardian 3.3 is a fallback with weaker BYOC.
- **The "physics may be wrong" detector** is a research direction more than a closed problem. The recommendation above is implementable in 12 days but is not a paper-ready result; the NeurIPS workshop paper should frame it as a contribution rather than a solved problem.
- **MLPerf-style reproducibility for a 12-day project is aspirational, not certain.** The realistic outcome is a reference Docker image + a CI test that the team itself uses, plus a public template for external reproduction. Independent third-party reproduction is unlikely to occur before the May 31 submission deadline.
- **Chronos-2 is a third-party API.** If it goes down or changes pricing during the 12-day build, the ensemble degrades to TTM + FlowState. Plan for that failure mode in advance.
- `CvxpyLayer` cannot be `torch.jit`-traced or `torch.compile`-d (per the cvxpylayers README); all inference must remain in eager mode.
- **Open question that needs a prototype to close:** whether unrolling 3 SCP iterations through cvxpylayers in DPP form is stable enough to converge inside an 8-tier Pacejka linearization on the RTX 4060. If a day-3 prototype shows oscillation or divergent linearizations, drop to 2 SCP iterations and add a soft trust-region penalty (per arXiv:2503.15373 and the trust-region-shrinking SCP literature in trajectory optimization).